

FACADE PATTERN

Structural Design Pattern

The Problem: Too Many Services to Talk To

A common real-world example of the **Facade Pattern** is an **API Gateway** in a microservices system.

Problem Without Facade:

Imagine you're building an e-commerce app (like Flipkart or Amazon). Your backend is split into multiple separate services:

- **User Service** – handles login, profile, etc.
- **Order Service** – handles placing orders
- **Inventory Service** – tracks product stock
- **Payment Service** – processes payments

Without a Facade (API Gateway), your mobile app or website would need to directly talk to ALL these services separately

The Problem: Too Many Services to Talk To

```
Mobile App
  └─> Call User Service API
  └─> Call Order Service API
  └─> Call Inventory Service API
  └─> Call Payment Service API
```

This creates problems:

- **Complex Client Code:** Your app needs to know about all 4 services and their different APIs
- **Tight Coupling:** If you change a service API, you must update the mobile app
- **Exposed Internals:** The app knows too much about your backend structure
- **Multiple Network Calls:** Slower performance with many API calls